A Small Guard in Front of the Controller: Adaptive HVAC Control with a Safety Guard for the NeurIPS 2026 Smart Buildings Competition

Abstract

Most building controllers only work on the building they were tuned for, so the energy waste in every other building goes untouched because retuning a controller for each new building costs more than the energy it would save. We describe a guarded adaptive controller for the NeurIPS 2026 Smart Buildings Competition that reads each new building from its metadata and live sensor stream at runtime and needs no retuning. It builds on the strongest shipped baseline, PID with hot water reset, with worst-zone tracking, a sensor band read from each zone's own setpoints, and occupancy-aware shutdown and pre-conditioning. Around it sits a small safety guard, ported from a policy engine for autonomous agents, that caps per-step setpoint changes, caps daily energy use, clips every action to a safe range, and falls back to the baseline when a proposal looks wrong, writing every intervention to a ledger. On the 30 shipped training scenarios, 8,640 steps each, the guarded policy scores a mean absolute error of 0.05604, against 0.05496 for the best baseline, 0.06550 for fixed setpoint, 0.06782 for PID, and 0.22453 for bang-bang. It has the lowest comfort regret of any policy tested, but it pays for that comfort in energy and carbon, and it fails worst in the coldest climates. The guard intervened 148,924 times, almost all of them small slew clamps.

1 Introduction

Buildings cause nearly 40 percent of carbon emissions in the United States, and heating, ventilation and air conditioning is 40 to 60 percent of that. A controller that saves energy in one building and wastes it in the next is not a solution, it is a tuning problem wearing a control problem's clothes. The NeurIPS 2026 Smart Buildings Competition asks for one policy, deployed unchanged, that controls six unseen buildings in five unseen climates. The policy sees each building's description and live sensor stream at runtime and must infer its strategy from those, with no chance to retune.

That is the problem this paper takes up, and the sentence that opens the abstract is the whole paper in one line. A learned controller trained on six buildings can easily become a seventh tuned controller, one that happens to have been tuned on six buildings at once. Our answer is deliberately simpler. We take the strongest shipped baseline, make it adaptive to what each building actually reports, and put a small guard in front of it that checks every action before it is issued. The guard does not learn. It enforces rules written down in advance, and it keeps a record of every time it had to step in.

The contribution is not a new learning algorithm. It is a transfer argument made concrete: a rule-based adaptive controller plus a policy guard, evaluated locally on all 30 training scenarios against all four shipped baselines, with the numbers reported honestly, including the places where the guard loses. The guarded policy trails the best baseline by 0.00108 in mean absolute error, about 2 percent, while cutting comfort regret by 36.4 percent and spending 63.4 percent more energy to do it. Outside the coldest climates it beats that same baseline on average. That trade is the finding.

2 The competition task

The task is the one defined by the competition paper and starter kit. Each scenario is a commercial building simulated for 30 days at 5 minute steps, 8,640 steps in all, in one of five climates: Atlanta, Denver, Fairbanks, Phoenix and Singapore. Every 5 minutes the policy chooses, per air handling unit, whether it runs and its supply-air temperature, static pressure and recirculation, and per zone its damper and reheat, plus the hot water supply temperature and pump pressure. The action dimensions change from building to building, so a policy cannot assume a fixed action vector. The simulator enforces ventilation floors, so switching everything off is not an energy strategy.

The reward at each step is r = 0.6 comfort - 0.2 energy cost - 0.2 carbon, where comfort is a regret, zero inside a zone's band, linear to 5 K outside it and quadratic and unbounded beyond, weighted by zone occupancy. The reward is never positive. The leaderboard metric is the mean absolute error of the mean per-step reward against zero, so lower is better and the score is the absolute value of the mean per-step reward. The training set is 6 buildings times 5 climate stations, 30 scenarios, and the evaluation set is 6 unseen buildings times 5 unseen stations, also 30 scenarios, run once each by the competition because the simulator is deterministic. Four baselines ship with the starter kit: bang-bang, fixed setpoint, PID, and PID with hot water reset, called pid_water here. No single baseline is best everywhere.

Round 1 of the competition is a poster paper, and this paper is that submission. No Kaggle notebook was submitted in round 1, so there is no private leaderboard score or notebook version URL to report. The score reported here is the local validation mean absolute error on the 30 training scenarios, computed with the shipped harness. The final round, in

January 2027, adds the Kaggle notebook and the held-out test set.

3 Method

The policy has two parts, an adaptive controller and a guard, and they do different jobs. The controller decides what to do. The guard decides whether that decision is allowed to reach the building. Nothing in either part is keyed to a building identifier, a floor plan, or a climate. Both read only the building metadata supplied at construction and the sensor stream supplied at each step.

3.1 The adaptive controller

The controller subclasses the shipped pid_water baseline and turns on two refinements the shipped chain leaves off. First, worst-zone tracking. The baseline drives each air handler from the average temperature of its zones, but the comfort penalty is unbounded in the worst zone's deviation. One zone 10 K out costs far more than ten zones 1 K out, and an average hides exactly that. This policy drives each handler from its worst zone instead. Second, a sensor band. The reward judges each zone against its own heating and cooling setpoint sensors, so this policy reads the band from those sensors instead of rebuilding it from the hour of day.

On top of that are two occupancy-aware behaviours. The comfort charge is weighted by occupancy, so conditioning an empty building is pure cost. When the building is unoccupied and comfort mode is not due within the hour, the air handlers shut down. When comfort mode is due within the hour, the controller pre-conditions against the comfort band instead of waiting for the band to snap over at the start of work hours. The policy is adaptive and rule-based, not a reinforcement learning agent and not a learned model. It was not trained on the six training buildings. It reads them at runtime.

3.2 The guard

The guard ports a policy engine built for autonomous agents to this setting. That engine has four rules, a per-action cap, a daily cap, an allowlist, and a confirm threshold, and the same four rules appear here, plus a ledger and one extra safety check. A shadow instance of the shipped pid_water baseline runs alongside on the same observation stream and supplies the safe fallback action.

The per-action cap is a slew rate. An air handler's supply-air temperature may move at most 0.5 normalized units per step, and only while the handler was already running and stays running. A cold start and a shutdown get their full move, because throttling those would fight the mode change instead of catching hunting. The daily energy budget cap lets the plant spend at most 80 percent of the maximum possible daily commanded effort in one simulated day. When the budget is spent, the guard forces eco mode, unless a zone is more than 5 K outside its band, in the quadratic penalty region, in which case comfort wins. The allowlist clips every continuous channel to the normalized range from -1 to 1, and the guard never heats a zone that is already hot: if a zone is above its cooling setpoint, its reheat valve is forced closed and its damper is forced to minimum. The confirm threshold replaces a handler's action with the safe baseline's action when a proposed supply-air temperature pushes against the measured zone error by more than 0.8 normalized units. The extra check is unsafe-off prevention: the proposal must not switch an air handler off while its own zones are outside their band, and an empty building with no comfort due is exempt.

Every intervention is appended to a JSONL ledger with the step, the day, the rule, the device, the field, the proposed and issued values, and a human-readable reason. A guard that cannot be audited is a guard that cannot be trusted, and the ledger is what makes the intervention counts in Section 5 exact rather than estimated.

Two rules were tried and removed, because each taught the same lesson. A minimum on-off time of 6 steps fired 47 times in a 288-step smoke test and interacted badly with the slew cap, holding a handler on at the coldest supply temperature and then throttling its recovery. A water-temperature slew cap of 0.3 per step fired 92 times in 288 steps, mostly with large clamps, because the shipped water reset switches the temperature between -1 when the loop is idle and the reset value when it is heating. A guard rule must not manufacture the violation the next rule has to catch, so both were removed rather than patched around. After both removals, a 288-step smoke test on building_train_1 in Fairbanks produced 27 interventions, all genuine air-handler slew clamps, and a total reward of -3.51, better than -3.98 with the water cap and -3.65 with the on-off hold.

4 Evaluation setup

Data sources. The data are the 30 shipped training scenarios, defined by the starter kit's training set file: 6 buildings, each run once in each of the 5 climates, with no strip used twice. No outside data were used, and no real building data were used. The simulation environment is the shipped simulator and harness, run locally with the shipped evaluate script. The policies compared are the four shipped baselines and the guarded policy, all run locally with the same code, on the same 30 scenarios, one 8,640-step episode each.

That last point matters, because the shipped baseline numbers are stale. A calibration run reproduced one shipped scenario locally: pid_water on building_train_1 in Fairbanks gives a total reward of -1848.31 locally against -1828.90 shipped, about 1 percent, or a mean absolute error of 0.21392 locally against 0.21168 shipped. The simulator is deterministic, so a gap means the code changed after the

shipped returns were generated. The cause is in the kit's history: a fix to the simulation tracker, which had been querying occupancy with VAV zone identifiers and so creating fictitious populations and consuming random draws, landed after the initial commit that shipped the returns. Fixing it changed the random stream, and therefore the trajectories. The paper therefore compares local against local only. The shipped numbers are shown in Table 1 as a reference, with the gap stated, and they are not used for any claim.

Computing infrastructure. One full 8,640-step episode takes about 9.5 minutes on a single core on the machine used here, about 0.063 seconds per step. Five policies times 30 scenarios is 150 episodes, about 24 hours of single-core compute. Two workers ran in parallel, each using about 450 MB, for about 12 hours of wall time. There was no training setup and no learned model to train, so there are no training hyperparameters, no GPU, and no training compute to report. The policy itself costs one PID-style rule per step plus the guard checks, which is negligible next to the simulator.

5 Results

Table 1 gives the headline result. The guarded policy scores a mean absolute error of 0.05604, against 0.05496 for pid_water, 0.06550 for fixed setpoint, 0.06782 for PID, and 0.22453 for bang-bang. It trails the best baseline by 0.00108, about 2 percent. It beats fixed setpoint in 19 of 30 scenarios, PID in 23 of 30, and pid_water in 16 of 30. Its best scenario is 0.00722, on building_train_5 in Fairbanks, and its worst is 0.26904, on building_train_1 in Fairbanks.

Table 1: Mean absolute error of mean per-step reward, lower is better, over the 30 training scenarios. Shipped values are the stale returns bundled with the starter kit; local values are the comparison used in this paper.

Policy	Shipped	Local	Best	Worst
bang-bang	0.22520	0.22453	0.07283	0.30828
fixed setpoint	0.06592	0.06550	0.01709	0.20181
PID	0.06736	0.06782	0.01703	0.19294
PID + water	0.05414	0.05496	0.01689	0.21392
Guarded (ours)	-	**0.05604**	**0.00722**	0.26904

Table 2 breaks the reward into its parts and shows where the headline score comes from. The guarded policy has the lowest comfort regret of any policy, -319.60 against -502.35 for pid_water, a 36.4 percent improvement. It pays for that comfort in energy, 811.90 against 496.96, 63.4 percent higher, and in carbon, 650.24 against 370.17, 75.7 percent higher. The reward weights comfort at 0.6 and energy and carbon at 0.2 each, so the comfort gain almost, but not quite, pays for the energy cost. That is the whole story of the 2 percent gap: the policy buys comfort too expensively.

Table 2: Reward breakdown, mean per scenario over 8,640 steps. Comfort is total normalized productivity regret, energy and carbon are total normalized costs.

Policy	Comfort regret	Energy cost	Carbon
bang-bang	-1652.60	2621.37	2120.56
fixed setpoint	-484.84	759.40	615.72
PID	-606.69	619.57	490.36
PID + water	-502.35	496.96	370.17
Guarded (ours)	**-319.60**	811.90	650.24

The guard itself intervened 148,924 times across the 30 scenarios. Almost all of those, 148,921, were slew clamps on supply-air temperature. The daily budget tripped only 3 times, all of them in building_train_1 in Fairbanks. That split is informative. The slew cap is doing the everyday work of the guard, smoothing the controller's proposals step by step, while the budget cap almost never binds, which suggests the daily budget as set is too loose to matter, or that the controller rarely spends that fast. Either way, the ledger makes the claim checkable: the counts come from the guard's own records, not from a post-hoc estimate.

The per-scenario pattern is uneven in a way the mean hides. The guarded policy wins outright on many scenarios, including its best, 0.00722 on building_train_5 in Fairbanks, and 0.00919 on building_train_1 in Denver and 0.00966 on building_train_4 in Phoenix. It also loses badly on a few cold ones: 0.26904 on building_train_1 in Fairbanks, where even the best baseline scores 0.21392, and 0.15475 on building_train_8 in Fairbanks. Splitting by climate makes the pattern plain. Across the 6 Fairbanks scenarios the guarded policy averages 0.09679 against 0.07906 for pid_water. Across the other 24 scenarios it averages 0.04585 against 0.04893 for pid_water, so outside Fairbanks it beats the best baseline on average. The failure is specific, not general, and Section 6 takes it up.

6 Failure modes, fairness and robustness

The main failure mode is cold-climate heating. In Fairbanks the guarded policy spends heavily to hold comfort, and the energy and carbon cost outweighs the comfort it buys. The worst zone logic is part of the cause: driving from the worst zone is the right call when the penalty is unbounded in the worst deviation, but in a cold climate the worst zone is often far outside its band for a long time, and chasing it means running the plant hard for hours. Pre-conditioning adds to the same bill, warming the building before comfort mode starts, which is cheap in a mild climate and expensive in a cold one. The guard's budget cap should have caught this, but it tripped only 3 times, all in one scenario, so it is not binding where it is needed most. That is a design failure to

fix, not a result to excuse: the budget needs to bind earlier in cold weather, or to be set per climate from the building's own heating demand, rather than as a flat 80 percent of maximum effort.

Two smaller failure patterns are worth naming. In Singapore the guarded policy loses on three of six buildings, including 0.04639 against 0.02835 for pid_water on building_train_1 and 0.12187 against 0.10990 on building_train_5, which suggests the worst-zone logic also overreacts in hot, humid conditions. On building_train_5 in Atlanta it scores 0.06315 against 0.03076 for pid_water, its worst non-Fairbanks result, which shows the policy can lose even in a mild climate when its occupancy shutdown misjudges the return of comfort mode.

On fairness, the honest statement is that the data do not let us test it properly. The scenarios differ by building and climate, not by occupant group, and the simulator carries no occupant demographics, so there is no subgroup analysis to report. What can be said is that the benefit is not shared evenly across buildings: the policy helps most buildings and hurts the coldest ones, and a controller deployed unchanged across a portfolio would spend its energy budget where the climate is harshest. Any real deployment would need to check that pattern against who occupies those buildings, and this paper does not. On robustness, the policy was run once per scenario, which is the competition's protocol for a deterministic simulator, so there is no variance to report and no seed sensitivity to hide. The guard's rules are deterministic given the same observation stream, and the ledger makes every intervention reproducible from the code and the scenario files.

7 Generalization to unseen buildings

The argument for transfer rests on what the policy does not contain. It contains no building identifier, no floor plan, no climate label, and no per-building tuning. It reads the air handler to zone map, the action indices, and the water indices from the building metadata at construction, and it reads zone temperatures, occupancy, and comfort mode flags from the live sensor stream at every step. The setpoint band comes from each zone's own sensors, so a building with a different band is read correctly without being told. The guard's caps are in normalized units, so they mean the same thing on a large building and a small one. That is the whole transfer mechanism, and it is deliberately boring: the policy adapts because it asks the building, every step, rather than because it remembers one.

That argument has a limit, and it should be stated plainly. The local evaluation is on the 30 training scenarios only. The evaluation set of 6 unseen buildings in 5 unseen stations is run by the competition, not locally, so this paper reports no unseen-building score and claims none. What the training results do show is that the policy transfers across the six training buildings and five climates without retuning, beating the best baseline on average outside Fairbanks and losing to it inside Fairbanks. That is evidence for transfer across buildings of the same kinds, not proof of transfer to buildings of a new kind.

8 Limitations and further work

The first limitation is the energy bill. Closing the 2 percent gap means spending less to buy the same comfort, and the most direct lever is the one the failure analysis points to: a budget cap that binds in cold weather, and pre-conditioning tied to the measured heating demand when the outside air is far below the band. A second limitation is that the policy is rule-based, so it cannot learn a building's thermal response over days, and it will keep making the same expensive choice in the same conditions. A learned controller could do better here, and the guard is exactly what would make such a controller safe to try. A third limitation is scope: 30 training scenarios, one deterministic run each, no real building, and no test under sensor noise, missing sensors, or a changed occupancy schedule. Those are the tests the final round should add, along with the Kaggle notebook and the held-out test set results.

9 Conclusion

A small guard in front of a controller does not need to win on the headline score to be useful. It needs to make the controller transferable, auditable, and honest about what it costs. This policy transfers without retuning, because it reads each building at runtime. It is auditable, because every one of its 148,924 interventions is in a ledger with a reason. And it is honest about its cost, because the numbers show it buying the best comfort in the field with the highest energy bill, and failing worst where the weather is coldest. That is a real result, and it is the one the final round should build on: keep the guard, and teach the controller to spend less for the comfort it already buys so well.

Code and data availability

Code and ledgers for the guarded policy, the evaluation runner, and the aggregation script are public at https://github.com/tydavid5/guarded-hvac. The data are the 30 shipped training scenarios from the competition starter kit, and the results in Tables 1 and 2 are computed from the local harness output in that repository. No Kaggle notebook was submitted in round 1.

References

1. Goldfeder, J., et al. The NeurIPS 2026 Smart Buildings Competition. arXiv:2610.18511, 2026.
2. Smart Buildings 2026. Starter kit for the NeurIPS 2026 Smart Buildings Competition. https://github.com/smart-buildings-2026/starter-kit, 2026.
3. NeurIPS 2026 Smart Buildings Competition. Kaggle competition page. https://www.kaggle.com/competitions/neurips-2026-smart-buildings/, 2026.
4. Rebedea, T., et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. arXiv: 2310.10501, 2023.